

스터디 소개

저희 팀은 현대 컴퓨터 비전을 공부하기 위해, 기초적인 KNN 분류기부터 시작하여 FC신경망, CNN신경망을 학습하고, RNN과 LSTM등 다양한 아키텍처를 학습한 후 현재는 Transformer 기반의 아키텍처를 학습 중입니다. KNN과 FC, CNN을 직접 numpy와 pytorch로 구현해 보았고, 앞으로는 Transformer 기반의 image captioning을 실습할 예정입니다.

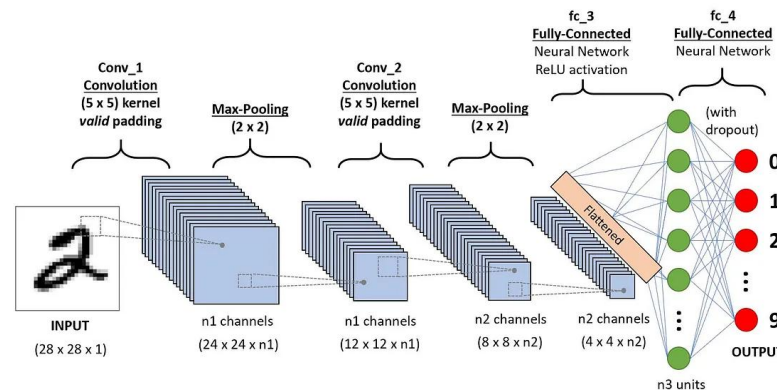
스터디는 각자 강의를 듣고 강의내용을 정리해 대면으로 만나 정리본에 대해 서로 피드백하고 질문하는 식으로 진행하였습니다.

주제 소개

Computer Vision은 검색, image understanding, 앱, 지도 제작, 의료, 드론, 자율주행차 등에 응용되며 우리 사회 어디에서나 찾아볼 수 있게 되었습니다. 이러한 응용 분야들의 핵심은 image classification, localization, detection과 같은 **visual recognition** 작업입니다. 최근 neural network (일명 "deep learning") 접근법의 발전은 이러한 state-of-the-art visual recognition 시스템의 성능을 크게 향상시켰습니다.

CS231n은 이러한 작업들, 특히 image classification을 위한 end-to-end 모델 학습에 초점을 맞추어 deep learning 아키텍처의 세부 사항을 깊이 있게 다룹니다. 강의를 수강하는 동안 저희들은 neural network를 직접 구현하고 학습하는 방법을 배우며, Computer Vision 분야의 최신 연구에 대해 자세히 이해하고자 합니다. 추가로 다양한 실습 과제를 통해, deep learning 작업을 세팅하기 위한 툴을 갖추고 deep neural network를 학습 및 fine-tuning 하기 위한 실전 엔지니어링 노하우를 습득하는 것을 목표로 합니다.

CNN



1. Motivation

일반 DNN의 한계: 이미지를 1차원으로 펼쳐서 입력하면 데이터의 형상(픽셀 간의 관계)이 파괴됨

CNN의 장점: 필터를 통해 구조적 정보를 유지하며 특징(엣지, 질감 등)을 학습

2. Core Architecture

Convolution Layer (특징 추출): 학습 가능한 필터(Kernel)가 이미지를 훑으며 특징 지도(Feature Map)를 생성

Pooling Layer (차원 축소): 특징 지도의 크기를 줄여 계산량을 감소시키고, 주요 정보만 남겨서 미세한 변화에 모델이 견고해지도록 도움

Fully Connected Layer (분류): 추출된 고차원 특징들을 하나로 펼친 후 분류

3. 주요 특징

Receptive Field: 필터가 한 번에 보는 영역의 크기로, 층이 깊어질수록 모델은 더 넓고 복잡한 형상을 보게 됨

Parameter Sharing: 같은 필터를 이미지 전체에 공유하여 학습해야 할 파라미터 수를 줄임

Assignment

```
def compute_distances_no_loops(self, X):
    """
    Compute the distance between each test point in X and each training point
    in self.X_train using no explicit loops.

    Input / Output: Same as compute_distances_two_loops
    """
    num_test = X.shape[0]
    num_train = self.X_train.shape[0]
    dists = np.zeros((num_test, num_train))
    #####
    Implement with Codex
    # TODO:
    # Compute the l2 distance between all test points and all training
    # points without using any explicit loops, and store the result in
    # dists.
    #
    # You should implement this function using only basic array operations;
    # in particular you should not use functions from scipy,
    # nor use np.linalg.norm().
    #
    # HINT: Try to formulate the l2 distance using matrix multiplication
    # and two broadcast sums.
    #####
    dists = np.sum(X ** 2, axis=-1).reshape(-1,1) + \
            np.sum(self.X_train ** 2, axis=-1).reshape(1,-1) - 2 * np.dot(X, self.X_train.T)
    return dists
```

CS231n은 강의영상과 더불어 실제 신경망을 직접 구현해보는 과제가 주어집니다. 첫번째 과제에서는 PyTorch등의 프레임워크 없이 NumPy만으로 Backpropagation 등의 신경망을 설계하는데 있어 필수적인 요소들을 직접 구현 해야 하며 강의 중반부 이후부터는 PyTorch를 사용하여 여러 신경망들을 직접 설계하게 됩니다.

Vision Transformer

